

SSL/TLS Termination with ALB and ACM

Amazon's Application Load Balancer (ALB) can *offload* TLS by terminating SSL/TLS on the load balancer. You provision an ACM (AWS Certificate Manager) certificate for your domain and attach it to an HTTPS listener on the ALB. The load balancer uses the ACM certificate's public/private key pair to decrypt incoming requests before routing them to back-end targets (Maintaining Transport Layer Security all the way to your container: using the Application Load Balancer with Amazon ECS and Envoy | Containers) (SSL certificates for your Application Load Balancer - Elastic Load Balancing). This allows your application instances to focus on business logic rather than encryption overhead. (By default the ALB-to-target connection is plain HTTP, since ALB must inspect HTTP headers for routing. You can choose to re-encrypt to the target, but that requires HTTPS back-end listeners and certificates on the targets.) For example, in Terraform you might create:

```
resource "aws_acm_certificate" "cert" {
  domain_name      = "example.com"
  validation_method = "DNS"
}

resource "aws_lb" "alb" {
  name            = "example-alb"
  load_balancer_type = "application"
  subnets       = var.public_subnets
}

resource "aws_lb_listener" "https" {
  load_balancer_arn = aws_lb.alb.arn
  port              = 443
  protocol          = "HTTPS"
  ssl_policy        = "ELBSecurityPolicy-2016-08"
  certificate_arn   = aws_acm_certificate.cert.arn
}
```

This creates an ALB with an HTTPS listener using the ACM certificate. ALB supports multiple certificates via SNI; the *default certificate* must match your domain name or wildcard (SSL certificates for your Application Load Balancer - Elastic Load Balancing). ACM manages public certificates for you, and will automatically renew DNS-validated certificates as long as the DNS validation CNAMEs remain in place (SSL certificates for your Application Load Balancer - Elastic Load Balancing) (Renewal for domains validated by DNS - AWS Certificate Manager).

Route 53 and DNS Internals (TLD, NS, CNAME, ALIAS, hosts, nsswitch)

DNS resolution involves multiple layers: root servers delegate to Top-Level Domain (TLD) servers (such as `.com` or `.net`), which in turn delegate to an authoritative nameserver for the specific domain. The authoritative server holds the DNS resource records (A, AAAA, CNAME, NS, etc.) for that domain (What is DNS? | How DNS works | Cloudflare) (What is DNS? | How DNS works | Cloudflare). For example, a query for `www.example.com` is routed from a root resolver to the `.com` TLD server, then to the nameservers specified by the NS records of `example.com`. A CNAME record creates an alias to another hostname (which may reside in the same or another domain), whereas a Route 53 **Alias record** is a special AWS feature that acts like a CNAME but can target AWS resources (such as an ALB, CloudFront, or another record in the same hosted zone) and even be placed at the zone apex (root) (Choosing between alias and non-alias records - Amazon Route 53) (Routing traffic to an ELB load balancer - Amazon Route 53). For instance, using Terraform:

```
resource "aws_route53_zone" "primary" {
  name = "example.com"
}

resource "aws_route53_record" "alias_to_alb" {
  zone_id = aws_route53_zone.primary.zone_id
  name     = "www"
  type     = "A"
  alias {
    name           = aws_lb.alb.dns_name
    zone_id        = aws_lb.alb.zone_id
    evaluate_target_health = true
  }
}
```

Here the `www` record is an Alias “A” to the ALB. (If you used a CNAME record instead, you could not use it at the zone apex, and Route 53 would charge for additional queries.) (Choosing between alias and non-alias records - Amazon Route 53) (Routing traffic to an ELB load balancer - Amazon Route 53).

On each EC2 instance or Linux host, name resolution is governed by `/etc/nsswitch.conf` and optionally `/etc/hosts`. By default most systems have a line like `hosts: files dns` in `nsswitch.conf`, meaning the resolver checks the local `/etc/hosts` file first, then DNS (Agilent | Using “nsswitch” to Use Both `/etc/hosts` and a Name Server). This allows you to override DNS on a specific machine (e.g. for testing). The domain name resolution order and method (files vs DNS vs LDAP, etc.) is fully controlled by `nsswitch.conf` (Format of the `nsswitch.conf` File (System Administration Guide: Naming and Directory Services (DNS, NIS, and LDAP))) (Agilent | Using “nsswitch” to Use Both `/etc/hosts` and a Name Server).

Domain Registration and Mapping to ALB

When you register a domain in Route 53, AWS automatically creates a public hosted zone and assigns four nameservers (NS records) to it (Adding or changing name servers and glue records for a domain - Amazon Route 53). It then updates the domain registration to use those name servers, so Route 53 is your DNS provider. If you register the domain elsewhere, you must manually create a hosted zone in Route 53 and then update your registrar to point to the four NS names. Once the hosted zone exists, you map the domain (or subdomain) to your load balancer via a Route 53 record. As noted, use an Alias record of type “A” for the ALB. This method supports root domains (e.g. `example.com`) and is free of DNS query charges (Routing traffic to an ELB load balancer - Amazon Route 53). You typically create one alias record for the apex and optionally `www`, both pointing to the ALB’s DNS name (which you can find in the AWS console or via API). For example:

```
# Example: mapping example.com apex to an ALB
resource "aws_route53_record" "root_alias" {
  zone_id = aws_route53_zone.primary.zone_id
  name     = "example.com"
  type     = "A"
  alias {
    name            = aws_lb.alb.dns_name
    zone_id         = aws_lb.alb.zone_id
    evaluate_target_health = true
  }
}
```

Behind the scenes, Route 53 creates an A record alias to the ALB’s target group, handling the dynamic IPs of the load balancer. (If using an external registrar, you would copy the `ns-*.awsdns.org` NS records from the Route 53 zone into the registrar’s configuration.)

Amazon S3 Public Access, ACLs, and Node.js API Usage

Amazon S3 buckets are **private by default**. AWS provides a “Block Public Access” setting at both account and bucket level to disable any public policies or ACLs from granting access (Blocking public access to your Amazon S3 storage - Amazon Simple Storage Service). When enabled, it prevents any bucket policy or object ACL from allowing public access, helping ensure data remains private. S3 still retains legacy Access Control Lists (ACLs) on each bucket/object, but ACLs are considered a *legacy* mechanism. The AWS security team explicitly recommends using IAM or bucket policies instead of ACLs, as ACLs are less centralized and more error-prone (IAM Policies and Bucket Policies and ACLs! Oh, My! (Controlling Access to S3 Resources) | AWS Security Blog). (New S3 buckets now have ACLs disabled by default.)

To access S3 in Node.js, use the AWS SDK. For example, uploading an object

with public-read ACL:

```
const AWS = require('aws-sdk');
const s3 = new AWS.S3({ region: 'us-east-1' });
await s3.putObject({
  Bucket: 'my-bucket',
  Key: 'files/hello.txt',
  ACL: 'public-read',
  Body: 'Hello, world!'
}).promise();
```

This will upload the object and set its ACL to public-read. To retrieve:

```
const data = await s3.getObject({ Bucket: 'my-bucket', Key: 'files/hello.txt' }).promise();
console.log(data.Body.toString());
```

The AWS SDK automatically signs requests with IAM credentials. Make sure any bucket policy or ACL does not conflict with your “Block Public Access” settings. In practice, one might keep S3 access restricted via IAM roles, and only grant public-read to specific objects (like website assets).

AWS Secrets Manager and Container Environment Injection

AWS Secrets Manager securely stores API keys, database passwords, etc., and rotates them if configured. To retrieve a secret via the CLI:

```
aws secretsmanager get-secret-value --secret-id my-secret --query SecretString --output text
```

This returns a JSON string of key-value pairs. One common pattern is to use `jq` to convert that JSON into shell `KEY=VALUE` lines and export them. For example, a Docker container’s entrypoint script might include:

```
if [ -n "$AWS_SECRET_ID" ]; then
  aws secretsmanager get-secret-value --secret-id "$AWS_SECRET_ID" \
    --query SecretString --output text | jq -r 'to_entries|map("\(.key)=\(.value|tostring)")' \
    > /tmp/secrets.env
  eval $(sed 's/^/export /' /tmp/secrets.env)
  rm -f /tmp/secrets.env
fi
```

This approach (from [Store app secrets on ENV vars using AWS Secrets Manager and Terraform](#). | Obytes)) fetches the secret JSON, uses `jq` to convert it to lines like `DB_USER=user1`, and then exports them, so the container process sees them as environment variables. In Terraform or ECS task definitions, you can pass the secret ID as an environment variable (`AWS_SECRET_ID`), and include the AWS CLI and `jq` in the container image. (Alternatively, ECS and other services support direct Secrets Manager integration to inject secrets into container environments.)

Amazon RDS Read Replicas, Lambda Promotion, and ASG Integration

Amazon RDS can create **read replicas** of a primary database to offload read traffic or to achieve cross-region redundancy. A read replica is a copy of the primary DB instance that is by default read-only, asynchronously replicating updates from the primary (AWS Lambda: Promoting RDS read replica on cross-region using AWS LAMBDA | by Hrishikesh Nangare | Ankercloud Engineering | Medium). If the primary database fails, you can *promote* a replica to become the new primary. This can be done manually (via console/CLI) or automated with Lambda. For example, a Lambda function (triggered by CloudWatch alarms or database events) could call:

```
rds = boto3.client('rds')
rds.promote_read_replica(DBInstanceIdentifier='my-replica-id')
```

This AWS API call makes the read replica writable, and after promotion it accepts new writes. The Medium blog example describes a cross-region failover: if the primary in us-east-1 fails, a Lambda in us-east-1 is triggered (via CloudWatch logs/alarms) to promote the ap-south-1 replica to primary (AWS Lambda: Promoting RDS read replica on cross-region using AWS LAMBDA | by Hrishikesh Nangare | Ankercloud Engineering | Medium). After promotion, one would typically create a new read replica of the new primary in the original region to restore redundancy.

Application servers (for example in an Auto Scaling Group) must then be updated to use the new primary endpoint. If using RDS custom endpoints (e.g. `mydb.cluster-XYZ.region.rds.amazonaws.com` for Aurora), this may be automatic. For single-instance RDS, one strategy is to store the DB endpoint or credentials in a config or Secrets Manager entry, which the application reads on startup. After a failover, you could update that secret (e.g. via another Lambda) so that new EC2 instances in the ASG pick up the new hostname. In short, the ASG can respond to RDS failover by redeploying or reconfiguring instances with the updated DB endpoint.

ACM and Certificate Management

AWS Certificate Manager (ACM) manages TLS certificates for AWS services. When requesting a certificate, specify the domain name (FQDN) and optionally a wildcard. For example, `*.example.com` covers `foo.example.com` and `bar.example.com` but not `foo.bar.example.com` (SSL certificates for your Application Load Balancer - Elastic Load Balancing). ACM supports DNS validation (preferred) or email validation. DNS validation involves creating a Route 53 CNAME record under your domain, which ACM then checks (SSL certificates for your Application Load Balancer - Elastic Load Balancing) (Renewal for domains validated by DNS - AWS Certificate Manager). Once issued, ACM automatically renews the certificate before expiry. For DNS-validated certs, ACM

will attempt renewal 60 days before expiry as long as the original validation CNAME records still exist (Renewal for domains validated by DNS - AWS Certificate Manager). If renewal cannot be completed, ACM emits EventBridge events at intervals (45, 30, 15, 7, 3, 1 days) before expiration (Renewal for domains validated by DNS - AWS Certificate Manager).

In Terraform you might manage ACM as follows:

```
resource "aws_acm_certificate" "cert" {
  domain_name          = "example.com"
  subject_alternative_names = ["www.example.com"]
  validation_method = "DNS"
}

resource "aws_route53_record" "cert_validation" {
  for_each = { for dvo in aws_acm_certificate.cert.domain_validation_options : dvo.domain_name }
  zone_id = aws_route53_zone.primary.zone_id
  name    = each.value.resource_record_name
  type    = each.value.resource_record_type
  records = [each.value.resource_record_value]
  ttl     = 60
}
```

This requests a cert for `example.com` and `www.example.com`, then creates the necessary DNS validation records. The Terraform loop handles one or more SANs. After validation, ACM issues the cert and you attach it to your ALB or CloudFront. Remember that ACM certificates are region-specific (except for CloudFront, which uses us-east-1).

Terraform Infrastructure-as-Code (Modules, Remote State, Reusability)

We use Terraform to define all AWS infrastructure as code. Key practices include **modularity** and **remote state management**. For example, separate modules might define the VPC, the ECS cluster, the RDS instances, etc. Using modules promotes reuse and clearer organization: you can, say, invoke a standard **network** module from the Terraform Registry to create subnets and route tables, then use another module for ECS services. In practice, a `main.tf` might include:

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "3.15.0"
  name    = "pilot-vpc"
  cidr    = "10.0.0.0/16"
  azs     = ["us-east-1a", "us-east-1b"]
  public_subnets = ["10.0.1.0/24", "10.0.2.0/24"]
}
```

This instantiates a VPC module which creates the VPC, subnets, internet gateway, etc. Modules can be published and versioned, enabling reuse across projects.

Terraform **state** is stored remotely (e.g. in an S3 bucket) with locking (DynamoDB) to allow collaboration. For example:

```
terraform {
  backend "s3" {
    bucket      = "my-terraform-state"
    key         = "dr-tf-pilot-light/terraform.tfstate"
    region      = "us-east-1"
    dynamodb_table = "terraform-lock"
    encrypt     = true
  }
}
```

Storing state remotely ensures that team members work from the same view of resources. You can share data between configurations using `terraform_remote_state` data sources if needed (Backend Type: `s3` | Terraform | HashiCorp Developer). However, HashiCorp recommends publishing outputs via dedicated means when possible, since `terraform_remote_state` gives access to the entire state (The `terraform_remote_state` Data Source | Terraform | HashiCorp Developer).

Each Terraform module should output key information (e.g. VPC ID, subnet IDs, etc.) and accept variables for flexibility. We generally avoid hardcoding values; instead we pass in parameters like CIDR blocks, instance sizes, etc. Terraform's plan/apply workflow ensures consistent, repeatable provisioning of the described architecture.

Amazon ECS Fargate Deployment with Docker

For containerized microservices, we use AWS ECS with the Fargate launch type (serverless containers). In Fargate mode, you do not provision EC2 instances; AWS runs each task in its own isolated environment. A typical setup:

1. **Dockerize the application.** Build a Docker image (e.g. Node.js or Python app) and push it to Amazon ECR (Elastic Container Registry).
2. **Task Definition.** Define an ECS task with `requires_compatibilities` = `["FARGATE"]`, `network_mode` = `"awsvpc"`, and CPU/memory. Specify the container image and port mappings. For example, Terraform code:

```
hcl    resource "aws_ecr_repository" "app" {      name = "myapp"
}    resource "aws_ecs_task_definition" "app" {      family
= "myapp"      requires_compatibilities = ["FARGATE"]      network_mode
= "awsvpc"      cpu                    = "256"      memory
= "512"      container_definitions = jsonencode([ {      name
= "myapp-container"      image      = "${aws_ecr_repository.app.repository_url}:latest"
essential = true      portMappings = [ { containerPort = 80,
```

```
protocol = "tcp" }]]      logConfiguration = {      logDriver
= "awslogs"      options = {      awslogs-group      =
"/ecs/myapp"      awslogs-region      = "us-east-1"
awslogs-stream-prefix = "ecs"      }      }      ]])      }
```

This defines an ECS task whose container listens on port 80 and sends logs to CloudWatch.

3. **ECS Service with ALB.** Create an ECS service of type FARGATE, in a security group that allows inbound traffic from the ALB. Attach it to an ALB target group:

```
resource "aws_lb_target_group" "app" {
  name      = "myapp-tg"
  port      = 80
  protocol  = "HTTP"
  vpc_id    = module.vpc.vpc_id
  target_type = "ip" # for awsvpc mode, use IP target type ([Use an Application Load
}

resource "aws_ecs_service" "app" {
  name            = "myapp-service"
  cluster         = aws_ecs_cluster.main.id
  task_definition = aws_ecs_task_definition.app.arn
  launch_type     = "FARGATE"
  desired_count   = 2
  network_configuration {
    subnets      = module.vpc.public_subnets
    security_groups = [aws_security_group.app.id]
    assign_public_ip = true
  }
  load_balancer {
    target_group_arn = aws_lb_target_group.app.arn
    container_name   = "myapp-container"
    container_port    = 80
  }
}
```

Note that for Fargate tasks using `awsvpc` mode, the target type must be `ip` (each task gets an ENI) (Use an Application Load Balancer for Amazon ECS - Amazon Elastic Container Service). The ALB listener (created earlier) should forward to this target group.

In summary, ECS Fargate simplifies deployment: you package the app as a container image, define the desired state in Terraform, and AWS handles the rest. Security groups control access (e.g. ALB SG allows 443/80 from the internet; container SG allows 80 from the ALB). Scaling is managed by adjusting `desired_count` or using an ECS Service Auto Scaling policy.

Route 53 Health Checks and Failover Routing

Route 53 supports health checks and DNS failover to improve availability. A Route 53 health check can monitor an endpoint (HTTP/S, TCP, or even CloudWatch metric) and report Healthy/Unhealthy. In Terraform:

```
resource "aws_route53_health_check" "hc" {
  fqdn          = "app.example.com"
  type          = "HTTPS"
  resource_path = "/status"
  request_interval = 30
  failure_threshold = 3
}
```

This periodically sends a request to `https://app.example.com/status`. You then configure DNS records with a *failover routing policy*. For example, create a PRIMARY and a SECONDARY record for the same name:

```
resource "aws_route53_record" "primary" {
  zone_id      = aws_route53_zone.primary.zone_id
  name         = "app.example.com"
  type        = "A"
  set_identifier = "Primary"
  failover    = "PRIMARY"
  alias {
    name           = aws_lb.alb.dns_name
    zone_id       = aws_lb.alb.zone_id
    evaluate_target_health = true
  }
  health_check_id = aws_route53_health_check.hc.id
}

resource "aws_route53_record" "secondary" {
  zone_id      = aws_route53_zone.primary.zone_id
  name         = "app.example.com"
  type        = "A"
  set_identifier = "Secondary"
  failover    = "SECONDARY"
  alias {
    name           = aws_lb.standby.dns_name
    zone_id       = aws_lb.standby.zone_id
    evaluate_target_health = true
  }
}
```

Here, DNS queries will go to the **primary** ALB under normal conditions. If the health check fails, Route 53 will start returning the **secondary** record (pointing to the secondary ALB or backup site) (Failover routing - Amazon Route 53). This implements an active-passive failover. (You can also use active-active by

giving both records weight or using latency-based policies, but failover policy is simplest for single failover scenarios.)

In addition, Route 53 health checks can send SNS notifications or CloudWatch alarms when a failure is detected. This ensures that you and your team are alerted if the primary endpoint goes down.

Observability via Amazon CloudWatch

We rely on Amazon CloudWatch for monitoring logs, metrics, and alarms across the stack. Key points:

- **Metrics:** All AWS services emit CloudWatch metrics. For ECS (Fargate), CloudWatch automatically collects CPU and memory utilization per service and task (Monitor Amazon ECS using CloudWatch - Amazon Elastic Container Service). Enabling CloudWatch Container Insights provides more granular container-level metrics (for a small additional cost). For RDS, AWS publishes many useful metrics like CPUUtilization, FreeStorageSpace, ReadIOPS, etc. under the `AWS/RDS` namespace (Amazon CloudWatch metrics for Amazon RDS - Amazon Relational Database Service) (Amazon CloudWatch metrics for Amazon RDS - Amazon Relational Database Service). For ALB, metrics include RequestCount, HTTP 4xx/5xx, and TargetResponseTime. We can create CloudWatch alarms on thresholds (e.g. CPU > 80%, or ALB 5xx errors > 1). Alarms can trigger SNS notifications, Lambda functions, or auto-scaling actions.
- **Logs:** We send application and container logs to CloudWatch Logs. In the ECS task definition above, `logDriver=awslogs` sends stdout/stderr of the container to a `/ecs/myapp` log group. Lambda functions (for RDS promotion, or others) also log to CloudWatch by default. RDS instances can export slow-query or general logs to CloudWatch Logs as well. Having logs centrally in CloudWatch allows searching and creating metric filters.
- **Dashboards and Insights:** We build CloudWatch Dashboards combining ECS, RDS, and ALB metrics to get an overview of the system's health. We can use Log Insights queries to analyze patterns in logs. In practice, having end-to-end observability means: if a deployment goes wrong or a replica fails over, we can trace the events in CloudWatch Events (EventBridge), metrics, and logs.

Overall, CloudWatch provides a single pane to monitor resource utilization, detect failures, and trigger automated remediation (e.g. a Lambda to promote an RDS replica on error). It is integral to our DevOps pipeline, replacing any self-hosted monitoring solution.